Personal Work-Experience Assistant

Updated 120-minute MVP implementation plan

1. Objective

Build a personal assistant that answers questions about professional experience using the CV as the initial authoritative source. The MVP prioritizes clean boundaries, grounded answers, provider independence, and automated evaluation.

2. Final MVP Architecture

Runtime: CV → ingestion → structured knowledge.md → CV retriever → RetrievedContext[] → LLM → answer + sources

Evaluation: golden dataset → DeepEval → real assistant → correctness / faithfulness / relevance + failure analysis

Main workflow: `assistant ingest` → regenerate knowledge → run evaluation → report failures

3. Knowledge Representation

Use structured Markdown rather than SQLite, FTS5, embeddings, or a vector database. The professional knowledge corpus is small, so simple, inspectable files are sufficient for the MVP. Organize knowledge into meaningful sections rather than arbitrary chunks.

```
# Coda
## Experimentation Platform
- Led backend infrastructure for experimentation.
- Built and operated ClickHouse infrastructure.
- Platform processed up to 300M events/day.
## Data Hub
- Led architecture of Coda Data Hub.
- Processed up to 12M events/day.
```

4. Retrieval Abstraction

Do not force CV, LinkedIn, GitHub, and blog posts into one storage model. Each source can own its ingestion/retrieval strategy. Normalize only at the retrieval boundary.

```
interface RetrievedContext {
  content: string;
  source: { type: string; id: string };
  metadata?: Record<string, unknown>;
}

interface Retriever {
  retrieve(query: string): Promise<RetrievedContext[]>;
}
```

MVP: `CvRetriever`. Future: additional source retrievers and/or a MultiSourceRetriever.

5. LLM Abstraction

Keep the model provider behind a small interface. Use Ollama locally for the MVP, but make the assistant independent of the provider.

```
interface LLM {
  generate(request: GenerateRequest): Promise<GenerateResponse>;
}
```

6. Assistant Runtime

```
User question
↓
Retriever.retrieve(question)
↓
RetrievedContext[]
↓
LLM: system instructions + context + original question
↓
Answer + provenance
```

The model should be instructed to stay grounded in supplied knowledge and explicitly say when the information is insufficient. Return or retain the retrieved context so answers can be inspected.

7. Golden Evaluation Dataset

Build at least 20 questions; target 25 for stronger coverage. Cover all five required categories:

Category	Target	Purpose
Directly answerable	8	Basic factual retrieval
Requires ≥2 sources	4	Tests cross-source synthesis
Ambiguous / underspecified	4	Tests clarification / cautious answers
Not answerable	4	Tests explicit fallback
Adversarial / hallucination / injection	5	Tests robustness and grounding
Total	25	

Prefer cases that define what an answer must contain and/or must never claim. Include a deliberately difficult question rather than only easy factual questions.

```
{
  "id": "clickhouse-experience",
  "category": "direct",
  "question": "What experience do I have with ClickHouse?",
  "mustContain": ["experimentation platform", "300M events/day"],
  "mustNotClaim": ["built ClickHouse itself"]
}
```

8. Real Source Conflict — Important MVP Addition

To demonstrate a real source conflict, the evaluation fixture needs two sources. This does NOT mean building two polished ingestion pipelines.

Recommended approach: implement the CV as the real MVP source and add a tiny LinkedIn fixture (or other synthetic second source) containing one intentionally conflicting fact. This satisfies the architecture/evaluation requirement while keeping the time box realistic.

```
fixtures/
■■■ cv.md
■■■ linkedin.md

Example conflict:
```

```
CV: Senior Software Engineer at Coda
LinkedIn: Lead Software Engineer at Coda
```

Question: “What was my title at Coda in 2024?”

Expected behaviour: retrieve both sources, expose the disagreement, and refuse to arbitrarily select a fact unless an authority rule exists.

Example answer: “Your sources disagree. The CV lists Senior Software Engineer, while LinkedIn lists Lead Software Engineer. I can't determine which is authoritative from the available evidence.”

This is intentionally a **minimal second source**, not a full LinkedIn integration. The exercise allows synthetic fixtures where appropriate.

9. Source Attribution & Fallback

Every answer should expose the sources behind it. When the system cannot answer, it should say so clearly rather than hallucinating.

```
Answer:
You have extensive ClickHouse experience...

Sources:
[1] CV — Coda / Experimentation Platform
[2] CV — Coda / Data Hub
```

```
Answer:
I can't answer that from the available knowledge. I found no source supporting that experience.

Sources searched: CV
```

10. Evaluation Metrics

Metric	Purpose
Correctness	Does the answer match the expected answer?
Faithfulness	Is the answer grounded in retrieved source material?
Relevance	Does it answer the user's question?

Use DeepEval where practical, but keep the evaluation cases and expected behaviour independent of the evaluation framework. The golden dataset is the contract; DeepEval is the measurement mechanism.

11. Handling Source / CV Changes

The golden dataset should not be silently rewritten when source material changes. Ingestion regenerates knowledge, then evaluation runs against it.

```
Source changes → ingest → new knowledge → evaluation
→ PASS: continue
→ FAIL: inspect whether the knowledge or golden expectation changed
```

Example: a metric changes from 12M/day to 20M/day. A failure may correctly indicate that an old golden expectation is stale. The developer reviews and approves the updated expectation.

12. Future Dynamic Golden Dataset Workflow

```
ingest → evaluate → identify failures → LLM diagnosis → suggested golden update → human approval →
update golden.json → rerun → commit
```

Future LLM assistance should propose changes and explain why a case appears stale; it should not silently mutate the golden dataset.

13. Future Data Sources

Source	MVP	Future value
CV	Yes	Curated career history and achievements
LinkedIn	Minimal fixture	Career enrichment / corroboration
GitHub	No	Projects, code, technical evidence
Blog posts	No	Technical reasoning and deeper context

The design supports new sources without redesigning the assistant: each source can implement its own retrieval logic and return RetrievedContext[].

14. Measured Operational Signals

Include two concrete numbers in the final README/report: measured p95 latency and rough cost per question.

```
Evaluation summary
Cases: 25
Passed: 21
Failed: 4
p95 latency: [measure this]
Cost / question: [measure / estimate this]
```

Do not invent these values. Measure them during the exercise and report the result honestly.

15. CLI

Command	Purpose
assistant ingest	Parse CV, regenerate knowledge.md, run evaluation
assistant ask "..."	Ask a question and display answer + sources
assistant eval	Run the golden evaluation independently

16. Suggested Project Structure

```
src/
■■■ assistant.ts
■■■ types.ts
■■■ ingestion/cv.ts
■■■ retrieval/cv.ts
■■■ llm/llm.ts
■■■ llm/ollama.ts
■■■ evaluation/eval.ts
■■■ cli.ts

knowledge/knowledge.md
eval/golden.json
fixtures/linkedin.md
```

17. Dependencies

Dependency	Purpose
commander	CLI
zod	Validation / structured data
ollama	Local Ollama client
vitest	Unit / integration testing
DeepEval	Evaluation metrics / LLM judging

If the CV is already Markdown/text, use Node.js built-ins for file handling. If it is a PDF, add a suitable parser. Verify the current DeepEval Node/TypeScript API before implementation.

18. 120-Minute Schedule

Time	Focus
0–10 min	Project skeleton + interfaces
10–35 min	CV ingestion → knowledge.md
35–60 min	Retriever + assistant + provenance
60–80 min	Ollama wrapper + fallback behaviour
80–105 min	25-case golden dataset + DeepEval
105–120 min	CLI, measured latency/cost, tests, README polish

19. Explicitly Out of Scope

Vector database • embeddings • SQLite/FTS5 • LangChain • LangGraph • full LinkedIn/GitHub/blog integrations • automatic golden mutation • knowledge versioning • complex agent orchestration • application-managed Git commits

20. Lead Engineer Narrative

The system deliberately chooses the simplest architecture that meets the problem. The corpus is small, so heavy retrieval infrastructure is not justified. Retrieval and LLM boundaries make future sources and model providers replaceable. Evaluation is a first-class part of the knowledge lifecycle. Source conflicts are surfaced rather than resolved by guesswork, and future LLM assistance proposes evaluation updates with human approval.

Target implementation: approximately 250–350 lines of clean TypeScript for the core MVP, with roughly 300–500 lines including CLI, evaluation, tests, and supporting code.